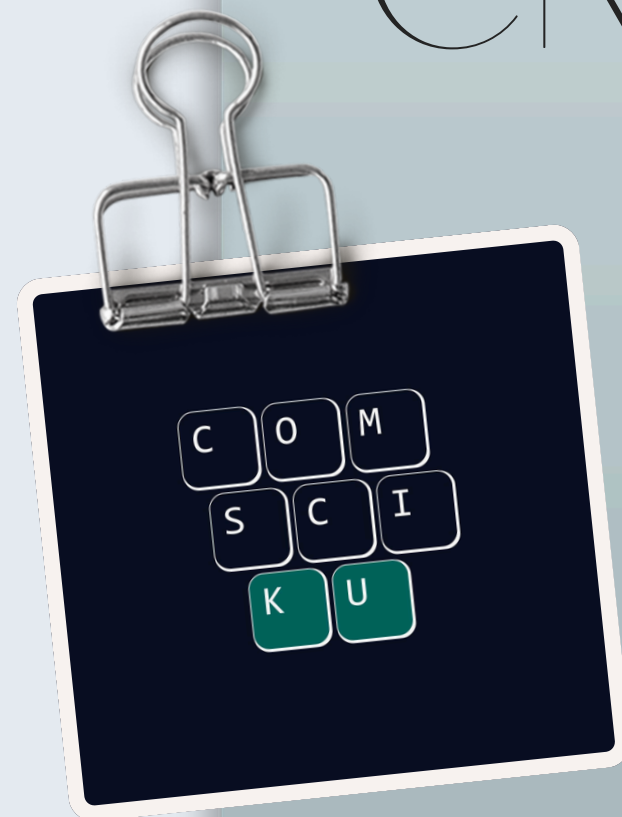
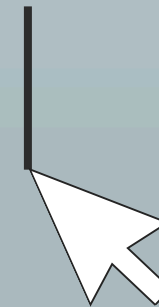


CNC ECONET ᄇ

RESILIENCE



PUMP





# CONTENT

HTTP TEST

SCRIPT TEST

SATURATION  
POINT

PACKET SIZE

REQUEST VS  
TIME

PACKET SIZE  
VS TIME



# HTTP TEST

คำสั่งที่ใช้ ปรับเปลี่ยนค่า user/request  
ab -n 5000 -c 25 http://172.19.0.5:8000/

ตัวอย่างค่าที่สรุปได้ ในแต่ละรอบการทดสอบ

Complete requests ทั้งหมด 10  
Failed requests 0  
Concurrency Level 1  
Requests per second 890 req/s  
Time per request 1.124 ms

[link to see test](#)

สรุปผลการทดสอบความทนทาน Resilience

1. การทดสอบระดับผู้ใช้งานต่ำ (1 User)

- **การทดสอบ 10 - 100 Requests:** ระบบสามารถจัดการได้สมบูรณ์แบบ ไม่พบ Request ที่ล้มเหลวเลย (Failed requests: 0)
- **Throughput (การรองรับต่อวินาที):** ทำได้ดีมาก โดยอยู่ที่ประมาณ 890 ถึง 1,222 requests/sec
- **Response Time (เวลาตอบสนอง):** ตอบสนองได้รวดเร็วมาก เฉลี่ยอยู่ที่ 0.818 ms ถึง 1.124 ms ต่อ Request

2. การทดสอบระดับผู้ใช้งานปานกลาง (100 - 1,000 Users)

- **ที่ 100 Users / 1,000 Requests:** ระบบทำ Throughput ได้สูงสุดที่ 3,618 requests/sec โดยมีเวลาตอบสนองเฉลี่ย (Average response time) อยู่ที่ 27.64 ms (ใช้เวลานานที่สุดเพียง 55 ms)
- **ที่ 1,000 Users / 1,000 Requests:** Throughput ตกลงมาเล็กน้อยอยู่ที่ 2,803.80 requests/sec ส่วนเวลาตอบสนองเฉลี่ยยังคงตัวอยู่ที่ 27.64 ms

3. การทดสอบระดับผู้ใช้งานสูง / โหลดหนัก (5,000 - 20,000 Users)

- การทดสอบมีการไล่ระดับ (Scale) การยิง Request จาก 5,000, 10,000, 20,000 ไปจนถึงจุดสูงสุดที่ 100,000 Requests (20,000 Users)
- **การใช้ทรัพยากร:** ที่โหลดระดับ 100,000 requests ทรัพยากรของระบบเป้าหมายทำงานหนักมาก โดยมีค่าระบุว่ากิน CPU และ Mem สูงถึง 96%

นำ kali ยิง http ซึ่งยังไม่ใช้ประเด็นสำคัญ

สิ่งที่พบเจอ ขอสรุปดังรูป

CPU ของ node นั้นๆ 96% กับ cloud 30%



# SCRIPT TEST

## ทดสอบ ECHONET LITE CONTROLLER

เพิ่ม REQUESTS เท่าไร CPU จะวิ่งประมาณ 90-110 % ไม่มากไปกว่านี้  
หากทางให้ระบบค้าง เด้ง ยังไม่ได้ ค่าที่ส่งเข้าไปอาจจะเล็กลงหรือเข้าไป

เป็นเพราะว่าตอนนี้วัดการส่ง ว่าส่ง PACKET ได้เร็วแค่ไหน

กำหนด 2G ที่ NODES-TV ด้วยคำสั่งนี้

DOCKER RUN --NAME=NODES-TV --MEMORY=2G ECHONET-  
NODES-TV

แต่ยัง SCRIPT เข้าไป ก็ไม่ได้ขยับ CPU MEMORY มากเท่าที่ควร

```
import socket
import time

TARGET_IP = "172.19.0.6"
PORT = 3610

REQUESTS = 100000

sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)

instance = 1

start = time.time()

for req in range(REQUESTS):

    tid = req & 0xFFFF

    packet = bytes([
        0x10, 0x81,
        (tid >> 8) & 0xFF,
        tid & 0xFF,
        0x02, 0x91, instance,
        0x05, 0xFF, 0x01,
        0x61,
        0x01,
```



# SATURATION POINT

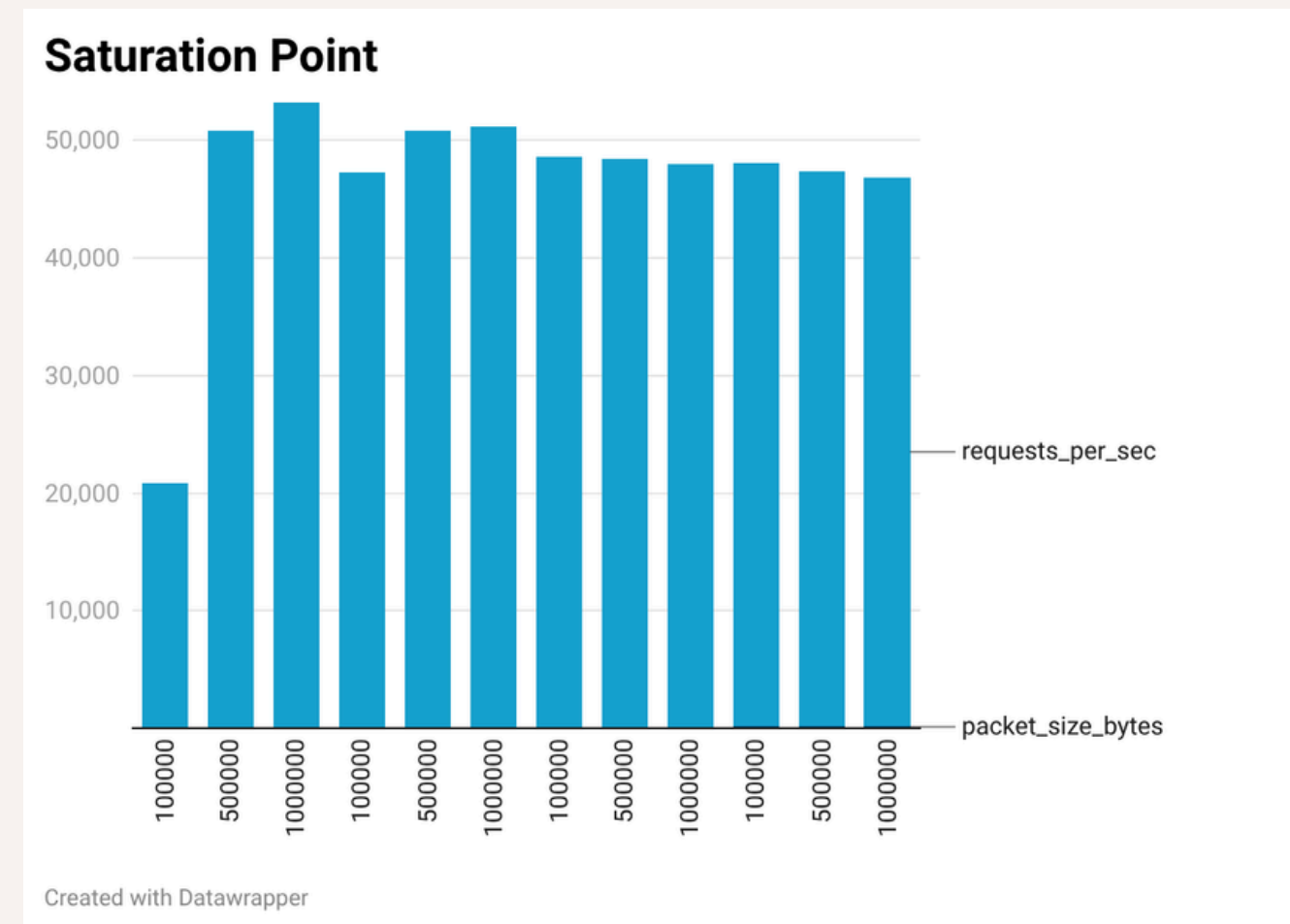
## หาขีดจำกัดของระบบ (SATURATION POINT)

ระบบมี “เพดานประสิทธิภาพ” ชัดเจน

ระบบสามารถรองรับได้ประมาณ: ~47,000 – 53,000 REQUESTS/SEC  
และจะ “ไม่เพิ่มขึ้นอีก” แม้เพิ่มโหลด

หมายความว่า: เพิ่ม REQUEST จาก 100K → 1,000,000 THROUGHPUT  
ไม่เพิ่มแล้ว ระบบเข้าสู่จุดอิ่มตัว (SATURATION)

**เพิ่ม REQUEST จาก 100K → 1,000,000  
THROUGHPUT ไม่เพิ่มแล้ว ระบบเข้าสู่จุดอิ่มตัว (SATURATION)**



โค้ดส่วนที่ COMMENT ใน GIT

ใช้เว็บนี้เอาข้อมูลไปสร้างกราฟ  
[HTTPS://WWW.DATAWRAPPER.DE/](https://www.datawrapper.de/)



# PACKET SIZE

## ขนาด PACKET แทนไม่มีผลต่อ PERFORMANCE

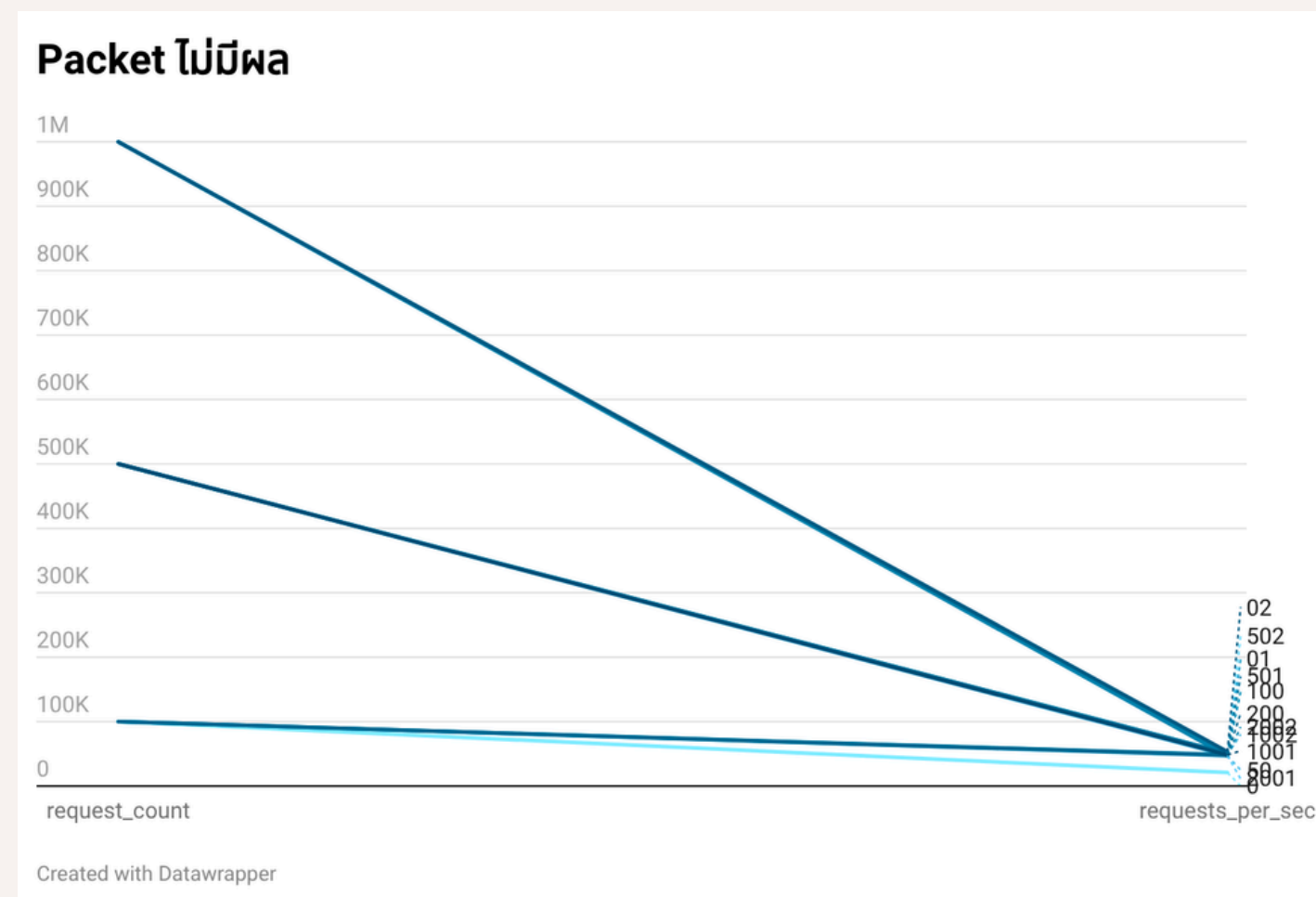
ทดสอบ PACKET SIZE:

0 BYTES  
50 BYTES  
100 BYTES  
200 BYTES

ผลลัพธ์:

THROUGHPUT แทนไม่เปลี่ยน (ยังอยู่ ~45K-53K RPS)

สรุปผล  
ระบบไม่ได้ติด BANDWIDTH แต่ติด PROCESSING / CPU / OVERHEAD





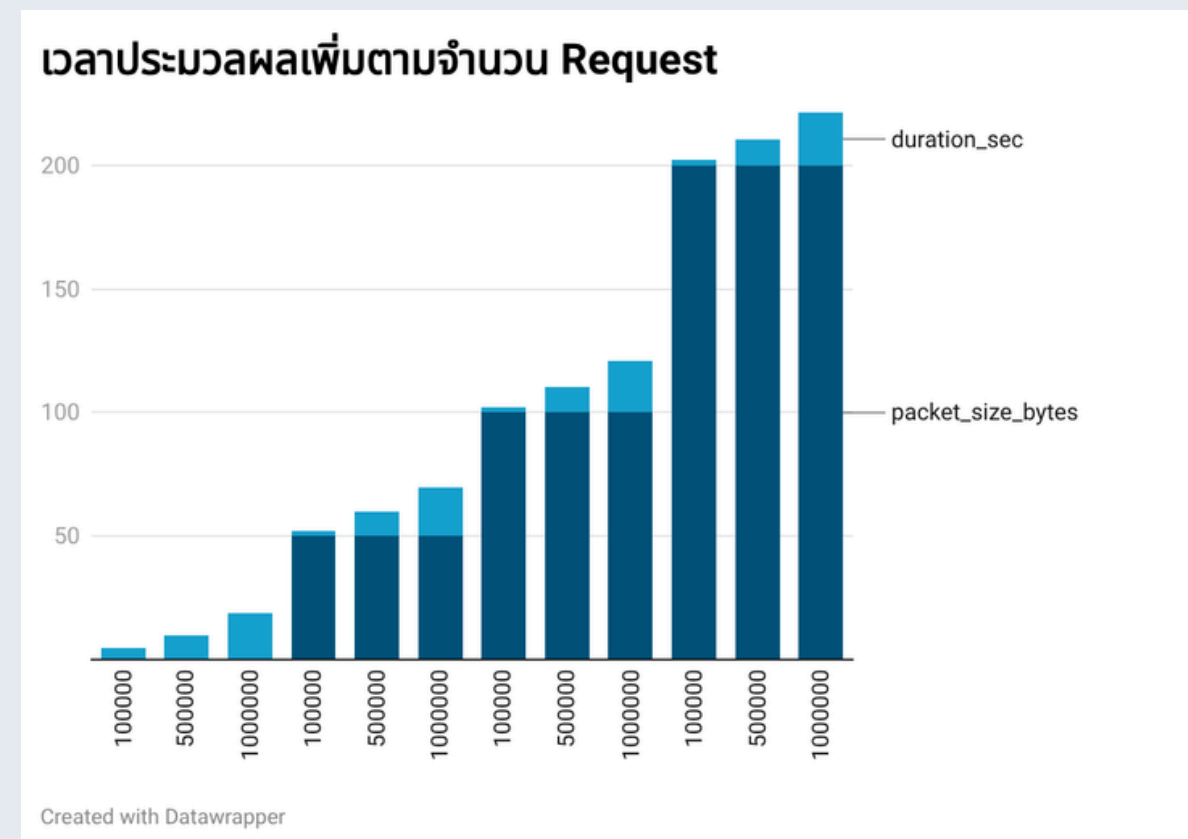
# REQUEST VS TIME

เวลาในการประมวลผลเพิ่มขึ้นตามจำนวน REQUEST อย่างชัดเจน

เมื่อเพิ่มจาก 100,000 → 10,000,000 REQUESTS เวลาประมวลผลเพิ่มจากประมาณ 2-5 วินาที เป็นประมาณ 200-220 วินาที

แสดงให้เห็นว่าระบบสามารถรองรับโหลดที่เพิ่มขึ้นได้ และเวลาตอบสนองเพิ่มขึ้นตามปริมาณงานที่ต้องประมวลผล

สรุปได้ว่า  
จำนวน REQUEST เป็นปัจจัยหลักที่ส่งผลต่อเวลาในการประมวลผล  
ของระบบ ECONET





# PACKET SIZE VS TIME

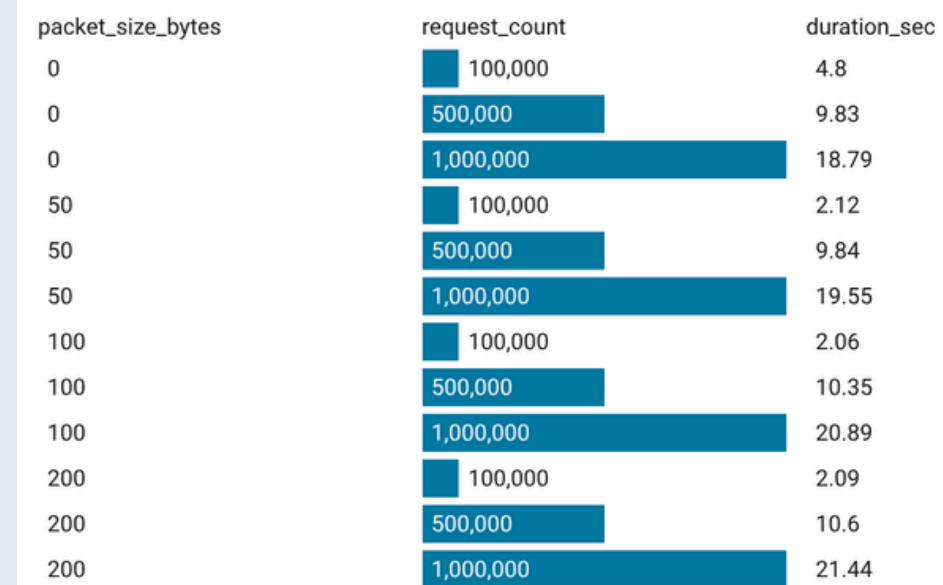
| 0 Bytes | 200 Bytes |
|---------|-----------|
| 4.80 s  | 2.09 s    |
| 9.83 s  | 10.60 s   |
| 18.79 s | 21.44 s   |

## ขนาด Packet มีผลต่อเวลาเพียงเล็กน้อย

- เมื่อใช้จำนวน Request เท่ากัน
- การเพิ่ม Packet จาก 0 → 200 Bytes
- เวลาในการประมวลผลเพิ่มขึ้นเพียงเล็กน้อย

จำนวน Request มีผลต่อเวลา มากกว่า  
ขนาด Packet มีผล ค่อนข้างน้อย ในช่วงที่ทดสอบ  
แสดงว่าคอขวดของระบบอยู่ที่ ปริมาณคำขอ (Request Processing) มากกว่าขนาดข้อมูลที่รับส่ง

### Packet ใหญ่ขึ้นทำให้เวลาเพิ่มขึ้นเล็กน้อย



Created with Datawrapper



# สรุปผล

| หัวข้อ       | Python UDP Script          | ApacheBench (ab)                                    |
|--------------|----------------------------|-----------------------------------------------------|
| โปรโตคอล     | UDP (ECHONET Lite)         | HTTP/TCP                                            |
| การเชื่อมต่อ | ไม่ต้องสร้าง Connection    | สร้าง TCP Connection และส่ง HTTP Request            |
| ใช้ทดสอบ     | ECHONET Controller         | Web Server / REST API                               |
| สิ่งที่วัด   | เวลาในการส่ง Packet, RPS   | Response Time, Throughput, Latency, Failed Requests |
| รอ Response  | ไม่รอ (ในโค้ดนี้)          | รอ Response ทุก Request                             |
| รูปแบบโหลด   | ส่ง Packet ตรงไปยังอุปกรณ์ | จำลองผู้ใช้เข้าเว็บพร้อมกัน                         |